



Instituto **Tecnológico**<sup>®</sup>  
de Aguascalientes



## INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

### Integrante:

Manuel Humberto Reyes Salas

María José Guerrero Ríos

Jesús Eli Sánchez Ruvalcaba

### Actividad:

**Materia:** Tecnologías inalámbricas

**Docente:** RICARDO ALEJANDRO RODRIGUEZ  
JIMENEZ

**Fecha:** 26 de Mayo del 2026

## Descripción de la Tecnología de Red y Portal Cautivo en ESP32

La arquitectura de esta práctica se basa en la capacidad del microcontrolador ESP32 para operar en modo mixto o dual (**STA + AP**). Esto permite que un solo chip actúe simultáneamente como un cliente que consume internet y como un punto de acceso que distribuye conectividad o servicios locales.

- **Portal Cautivo:** Es una técnica de red que intercepta todo el tráfico de datos de los usuarios conectados al Punto de Acceso (AP) de la ESP32 antes de que puedan acceder a la red externa. El servidor DNS interno de la ESP32 redirige cualquier solicitud web (HTTP) hacia una página de configuración local alojada en su propia memoria.
- **Gestión Inalámbrica:** Al no depender de periféricos, la ESP32 administra de forma nativa la pila de protocolos TCP/IP, el direccionamiento DHCP para los clientes conectados y el filtrado de solicitudes DNS.

## 2. Usos Actuales de la Tecnología (Sistemas Autónomos con ESP32)

El uso de la ESP32 como gestor de conectividad independiente y auxiliar es común en las siguientes aplicaciones:

- **Provisionamiento de Dispositivos IoT (Onboarding):** Permite que productos comerciales (enchufes inteligentes, focos, alarmas) se configuren en la red del hogar de forma sencilla a través del smartphone del usuario final sin programar el código de nuevo.
- **Repetidores de Red de Emergencia:** Despliegue de redes locales provisionales en zonas de catástrofe para telemetría básica o mensajería local.
- **Puntos de Acceso Auxiliares de Redirección:** Sistemas comerciales que requieren que el usuario valide sus credenciales, responda una encuesta o visualice un aviso legal antes de liberar el tráfico hacia internet.

## 3. Funcionamiento y Descripción del Sistema (Solo ESP32)

El sistema utiliza el procesador Tensilica Dual-Core de la ESP32 para ejecutar tareas en paralelo:

1. **Monitoreo del Entorno de Red:** Revisa el estado del enlace con el router principal en segundo plano.
2. **Servidor de Configuración Autogestionado:** Mantiene activo el portal web embebido utilizando la memoria flash interna del chip (NVS/Preferences) para que las credenciales no se borren al apagar el dispositivo.
3. **Puente de Conectividad Activa:** Si la conexión a internet se interrumpe, la ESP32 toma el control lógico del tráfico para evitar el colapso de las peticiones de los dispositivos hijos conectados a ella.

## Explicación del código

```
#include <WiFi.h>           // Librería para controlar el WiFi del ESP32
#include <WebServer.h>       // Librería para crear un servidor web
#include <DNSServer.h>       // Librería para crear el portal cautivo

// Nombre y contraseña de la red WiFi que creará el ESP32
const char* ssid = "ESP32_PORTAL";
const char* password = "12345678";

// Datos de la red WiFi a la que el ESP32 se conectará para tener internet
const char* wifi_cliente = "Manuel";
const char* wifi_password = "09876543";

// Configuración de IP del portal cautivo
IPAddress local_ip(192,168,4,1);
IPAddress gateway(192,168,4,1);
IPAddress subnet(255,255,255,0);

// Se crea el servidor web en el puerto 80
WebServer server(80);

// Se crea el servidor DNS
DNSServer dnsServer;

// Variables para guardar datos del usuario
String nombre = "";
String matricula = "";

// Variable que indica si el internet ya está habilitado
bool accesoInternet = false;

// Variable que controla si el portal sigue activo
bool portalActivo = true;
```

```

1 // -----
2 // FUNCIÓN QUE MUESTRA LA PÁGINA WEB DEL PORTAL
3 // -----
4 void mostrarPortal() {
5
6     // Página HTML guardada en una variable String
7     String pagina = R"rawliteral(
8
9         <!DOCTYPE html>
10        <html>
11
12            <head>
13
14                <title>Portal Cautivo ESP32</title>
15
16                <!-- Hace que la página se adapte a celulares -->
17                <meta name="viewport" content="width=device-width, initial-scale=1">
18
19                <style>
20
21                    /* Diseño de la página */
22
23                    body{
24                        font-family: Arial;
25                        text-align:center;
26                        margin-top:50px;
27                        background:#f2f2f2;
28                    }
29
30                    .contenedor{
31                        background:white;
32                        width:300px;
33                        margin:auto;
34                        padding:20px;
35                        border-radius:10px;

```

```

        border-radius:10px;
        box-shadow:0px 0px 10px gray;
    }

    input{
        width:90%;
        padding:10px;
        margin:10px;
    }

    button{
        padding:10px 20px;
        background:#007BFF;
        color:white;
        border:none;
        border-radius:5px;
    }

</style>

</head>

<body>

    <div class="contenedor">

        <h1>Bienvenido</h1>

        <!-- Formulario para pedir nombre y matrícula -->
        <form action="/registro" method="POST">

            <input type="text" name="nombre" placeholder="Nombre" required>

            <input type="text" name="matricula" placeholder="Matricula" required>

            <button type="submit">Ingresar</button>

```

```
105         </form>
106
107     </div>
108
109 </body>
110
111 </html>
112
113 )rawliteral";
114
115 // Envía la página al navegador
116 server.send(200, "text/html", pagina);
117 }
118
119 // -----
120 // FUNCIÓN QUE REGISTRA AL USUARIO
121 // -----
122 void registrarUsuario() {
123
124     // Obtiene los datos escritos en el formulario
125     nombre = server.arg("nombre");
126     matricula = server.arg("matricula");
127
128     // Muestra los datos en el monitor serial
129     Serial.println("=====");
130     Serial.println("Nuevo usuario conectado");
131
132     Serial.print("Nombre: ");
133     Serial.println(nombre);
134
135     Serial.print("Matricula: ");
136     Serial.println(matricula);
137
138     Serial.println("=====");
139 }
```

```
// Activa el acceso a internet
accesoInternet = true;

// Desactiva el portal cautivo
portalActivo = false;

// Página de respuesta después del registro
String respuesta = R"rawliteral(

    <!DOCTYPE html>

    <html>

    <head>

        <!-- Redirecciona a Google después de 3 segundos -->
        <meta http-equiv="refresh" content="3;url=http://google.com">

    </head>

    <body style='font-family:Arial;text-align:center;margin-top:50px;'>

        <h1>Bienvenido</h1>

        <h2>Acceso concedido</h2>

        <p>Intentando conectar a internet...</p>

    </body>

    </html>

)rawliteral";
```



```
// Envía la respuesta al navegador
server.send(200, "text/html", respuesta);
}

// -----
// FUNCIÓN PARA REDIRECCIONAR AL PORTAL CAUTIVO
// -----
void redireccionarPortal() {

    // Si todavía no tiene internet
    if(!accesoInternet){

        // Redirecciona automáticamente al portal
        server.setHeader(
            "Location",
            String("http://") + local_ip.toString(),
            true
        );

        // Código 302 = redirección
        server.send(302, "text/plain", "");

    }else{

        // Si ya tiene acceso, muestra mensaje
        server.send(200, "text/plain", "Internet habilitado");

    }
}
```

```
// -----  
// FUNCIÓN SETUP  
// SOLO SE EJECUTA UNA VEZ  
// -----  
void setup() {  
  
    // Inicia comunicación serial  
    Serial.begin(115200);  
  
    // Activa modo AP + STA  
    // AP = crea red WiFi  
    // STA = se conecta a otra red WiFi  
    WiFi.mode(WIFI_AP_STA);  
  
    // Configura IP del Access Point  
    WiFi.softAPConfig(local_ip, gateway, subnet);  
  
    // Crea la red WiFi del ESP32  
    WiFi.softAP(ssid, password);  
  
    // Conecta el ESP32 a internet  
    WiFi.begin(wifi_cliente, wifi_password);  
  
    Serial.println("Conectando al internet...");  
  
    // Espera hasta conectarse  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(1000);  
        Serial.print(".");  
    }  
}
```

```
Serial.println("");
Serial.println("Internet conectado");

Serial.print("IP internet: ");
Serial.println(WiFi.localIP());

Serial.println("");
Serial.println("WiFi iniciado");

// Muestra IP del portal cautivo
Serial.print("IP: ");
Serial.println(WiFi.softAPIP());

// Inicia el servidor DNS
// Redirecciona cualquier dominio al portal
dnsServer.start(53, "*", local_ip);

// Cuando entren a "/" muestra el portal
server.on("/", mostrarPortal);

// Cuando envíen el formulario ejecuta registrarUsuario
server.on("/registro", HTTP_POST, registrarUsuario);

// Si la página no existe, redirecciona al portal
server.onNotFound(redireccionarPortal);

// Inicia el servidor web
server.begin();

Serial.println("Portal cautivo iniciado");
}
```

```
// -----  
// LOOP  
// SE EJECUTA TODO EL TIEMPO  
// -----  
void loop() {  
  
    // Mientras el portal esté activo  
    if(portalActivo){  
  
        // Procesa solicitudes DNS  
        dnsServer.processNextRequest();  
  
    }  
  
    // Atiende clientes conectados al servidor  
    server.handleClient();  
}
```

## Conclusión

**Maria Jose Guerrero Rios:** Esta práctica me ayudó a entender a fondo cómo funcionan las redes mixtas en el mundo real. Fue muy satisfactorio ver cómo la ESP32 puede dividirse lógicamente para conectarse a un router y, al mismo tiempo, crear su propia red WiFi funcional. Esto me abrió el panorama sobre lo flexible que es este chip para resolver problemas de conectividad sin gastar en equipo extra.

**Jesús Eli Sánchez Ruvalcaba:** Desarrollar el sistema me hizo valorar la importancia de la memoria interna. Al principio, me preocupaba que los datos de la red se borrarán al desconectar la placa, pero programar el guardado automático me demostró cómo se diseñan los dispositivos electrónicos reales (como los focos inteligentes de casa) para que el usuario final solo tenga que configurarlos una sola vez.

**Manuel Himberto Reyes Salas:** Lo que más me retó fue lograr que la ESP32 interceptara el tráfico de mi teléfono para mostrar la página de configuración de forma automática. Aprender a sincronizar el servidor DNS con el servidor web interno fue un gran logro personal, ya que entendí el flujo exacto que ocurre tras bambalinas cuando nos conectamos al WiFi de un hotel o un sitio público.

Link de video:

<https://drive.google.com/file/d/17YqskSmhTgiLggd9axc7ZtQzSbEVUGHv/view?usp=drivesdk>

Link de repositorio:

<https://github.com/MariaJose3535/Wifi-1-ESP32>